

安全感知任务调度测试方案文档

1. 测试目标

本测试方案验证安全感知任务调度复现实验是否满足比赛交付要求，重点确认：

1. 代码可以在指定环境中编译或运行。
2. 三类实验入口可复现输出结果。
3. 输出指标覆盖功能任务可调度性与安全任务完整性。
4. 正常、异常和边界场景均有明确测试方法。
5. 测试过程可被评审者根据仓库代码独立复现。

2. 测试范围

测试对象包括：

- `rta/safety_aware_sched.py`：实验入口。
- `rta/util_syscall_cpu.py`：任务生成与测试集合来源。
- `rta/rta.py`、`rta/rta_omnilog.py`、`rta/rta_ellipsis.py`、`rta/rta_nodrop.py`：核心响应时间分析模块。
- `rta/params.py`：安全审计框架参数。
- 输出文件：`output/safety_aware_sched/*.txt`。

3. 测试环境

3.1 硬件环境

推荐环境：

- CPU：x86_64，4 核或以上。
- 内存：8 GB 或以上。
- 磁盘：至少 2 GB 可用空间。

最小环境：

- CPU：x86_64，2 核。
- 内存：4 GB。
- 磁盘：1 GB 可用空间。

3.2 软件环境

- 操作系统：Ubuntu 14.04/18.04 或兼容 Linux 发行版。
- Python：2.7.x。
- Python 依赖：NumPy、SciPy。

- 构建工具：GNU Make、SWIG 3.0、g++、libgmp。
- 第三方库：SchedCAT，位于 `lib/schedcat`。
- 可选：CPLEX，用于 SchedCAT 中需要线性规划求解器的实验。本复现实验主要依赖任务集生成和响应时间分析，仍建议按 README 配置。

3.3 网络配置

运行实验本身不需要网络。首次准备依赖时可能需要访问软件源或第三方依赖仓库。复现测试应在依赖安装完成后断网运行一次，以验证实验不依赖在线服务。

4. 测试工具选型

- `python`：执行实验入口和语法检查。
- `make`：构建 SchedCAT。
- `diff` 或人工比较：检查固定随机种子下输出数据是否一致。
- 文本检查工具：确认输出文件包含配置、环境、数据和运行信息。

5. 测试流程

5.1 环境准备

```
git submodule update --init --recursive
source setpath.sh
cd lib/schedcat
make
cd ../../
```

若 SchedCAT 构建提示缺少 LP Solver，按 README 设置 `CPLEX_PATH` 后重试。

5.2 快速冒烟测试

使用小样本快速验证入口可运行：

```
python rta/safety_aware_sched.py \
  --experiment utilization \
  --samples 2 \
  --util-min 0.5 \
  --util-max 0.52 \
  --util-step 0.01 \
  --output-dir output/safety_aware_sched_smoke
```

预期结果：

- 命令退出码为 0。
- 生成 `output/safety_aware_sched_smoke/utilization.txt`。
- 输出文件包含 `CONFIGURATION`、`ENVIRONMENT`、`DATA`、`RUN` 四段。

- `DATA` 中至少有 3 行数据，对应 0.50、0.51、0.52 三个利用率点。

5.3 完整复现实验

```
python rta/safety_aware_sched.py --experiment all --samples 1000
```

预期结果：

- 生成 `output/safety_aware_sched/utilization.txt`。
- 生成 `output/safety_aware_sched/syscall_count.txt`。
- 生成 `output/safety_aware_sched/cpu_count.txt`。
- 文件中记录随机种子、样本数、CPU 数、系统调用强度等参数。

6. 测试用例设计

6.1 正常场景

编号	场景	命令	评判标准
N1	利用率扫描	<code>--experiment utilization --samples 10</code>	输出利用率从 0.5 到 1.0 的可调度率
N2	系统调用强度扫描	<code>--experiment syscall_count --samples 10</code>	输出 <code>SYSCALL_COUNT</code> 横轴和各策略可调度率
N3	CPU 数量扫描	<code>--experiment cpu_count --samples 10</code>	输出 <code>CPU_COUNT</code> 横轴和各策略可调度率
N4	全量入口	<code>--experiment all --samples 10</code>	三个输出文件均生成

6.2 异常场景

编号	场景	操作	评判标准
E1	未构建 SchedCAT	清理或不构建 <code>lib/schedcat</code> 后运行	导入或运行阶段明确报错，不生成伪结果
E2	非法实验名	<code>--experiment unknown</code>	argparse 拒绝参数并返回非 0
E3	输出目录不存在	指定新目录	自动创建目录并写入结果
E4	样本数为 0	<code>--samples 0</code>	不建议作为有效评测；若执行，应检查是否有除零或空均值错误，并在评审中说明样本数需大于 0

6.3 边界场景

编号	场景	命令	评判标准
B1	单点利用率	<code>--util-min 0.75 --util-max 0.75 --samples 5</code>	输出 1 行数据
B2	最小 CPU	<code>--experiment cpu_count --cpu-min 1 --cpu-max 1</code>	输出 1 行数据
B3	高系统调用强度	<code>--experiment syscall_count --syscall-min 100 --syscall-max 1000 --syscall-step 100</code>	可调度率随负载变化，命令正常结束
B4	固定随机种子复现	连续运行两次相同命令	<code>DATA</code> 段数值一致，时间戳可不同

7. 性能指标与评判标准

主要评判指标：

- 调度可行率：每个横轴点下可调度样本数 / 总样本数，越高说明策略对功能任务 deadline 的保障越好。
- 事件驻留时间最大值：NoDrop 可调度样本中安全事件从产生到消费的最坏驻留时间，越低说明安全响应越及时。
- 事件驻留时间平均值：安全事件处理的平均延迟，用于比较不同配置下安全任务负载变化。
- 资源利用率变化趋势：利用率扫描中各策略从可调度到不可调度的拐点。

通过标准：

- 冒烟测试必须通过。
- 全量复现实验必须能生成三类输出文件。
- 输出文件必须包含完整配置与环境信息。
- 固定随机种子下，除时间戳和耗时外，`DATA` 段应保持一致。
- 不允许通过删除失败样本或跳过调度分析来提高可调度率。

8. 可复现部署说明

评审者可按如下步骤从干净仓库复现：

```
git clone <repo-url>
cd schedcat-experiments
source setpath.sh
cd lib/schedcat && make && cd ../../
python rta/safety_aware_sched.py --experiment all --samples 1000
```

若只需快速确认代码路径，可使用：

```
python rta/safety_aware_sched.py --experiment utilization --samples 2 --util-min 0.5 --util-max 0.51
```

9. 风险与后续验证

- 当前实现是算法实验复现版本，不直接修改 OpenHarmony 内核调度器。后续参赛提交若要求运行在 OpenHarmony 上，应将 `rta_nodrop` 的安全消费任务模型迁移为内核调度类或安全审计服务的调度策略。
- 原始仓库基于 Python 2.7，若迁移到 Python 3，需要处理 `itervalues`、旧式异常语法和 SchedCAT 兼容性问题。